ASSIGNMENT NO. 02

AIM: Implement notifications and background tasks using Android Services or iOS Background Modes for periodic or continuous tasks.

THEORY:
Asynchronous Data Processing and User Alerts in Android

1. Background Tasks: In Android development, background tasks are asynchronous operations that run completely independently of the application's main thread (the UI thread). Because network requests—such as fetching large JSON payloads from a News API—experience variable latency, executing them on the main thread would freeze the application interface. Background tasks solve this by offloading heavy processing, ensuring the app remains fluid and responsive to user input while data is retrieved and parsed behind the scenes.

2. Types of Background Tasks in Android: The Android OS provides distinct mechanisms for background execution, tailored to task priority:

- **WorkManager:** The modern, robust Jetpack solution for deferrable tasks. It is highly optimized for system health, automatically factoring in device state (doze mode, battery level, network type) before execution. It is the ideal choice for periodic content syncing, such as checking for breaking news.
- **Services:**
    - **Foreground Services:** Utilized for active, continuous tasks that the user initiates and must be visibly aware of (e.g., downloading a massive podcast file or tracking a GPS route). These strictly mandate an ongoing notification.
    - **Background Services:** Legacy components for invisible tasks, now heavily restricted by modern Android systems to prevent malicious battery drain.

3.Notifications: Notifications are transient, system-level UI elements that display outside of the application's standard visual context. They serve as the primary communication bridge between the background logic and the user, delivering critical alerts—such as breaking news updates—when the app is minimized, closed, or the screen is locked.

4. Notification Components: Constructing a modern Android alert requires the orchestration of two core APIs:

- **Notification Channel:** A mandatory grouping system (API 26+) that categorizes alerts. For a news app, developers can create distinct channels for "Technology," "Business," or "Sports," empowering the user to toggle specific categories on or off via system settings.
- **Notification Manager:** The centralized system service that acts as the publisher. It receives the constructed NotificationCompat.Builder payload (containing the icon, title, text, and expansion style) and renders it safely into the Android notification drawer.

5. Use Cases: Implementing background processing and alerts is crucial for:

- **Periodic Updates:** Silently querying a REST API every few hours to check if new articles have been published for a tracked keyword.

- **Data Sync:** Downloading thumbnail images or caching article text in local storage before the user even opens the app.
- **Reminders:** Notifying users of scheduled events or time-sensitive tasks.

6. Advantages:

- **User Engagement:** High-quality, personalized alerts (like notifying a user of an article matching their specific interests) significantly increase daily active app usage.
- **Real-Time Updates:** Eliminates the need for the user to manually refresh the application; the system pushes information proactively.
- **Battery-Efficient Execution:** By scheduling tasks intelligently rather than keeping the app awake via continuous polling, the application strictly adheres to Android's power-management guidelines.

CODE:

NewsWorker.kt:

```
class NewsWorker(appContext: Context, workerParams:
WorkerParameters) : CoroutineWorker(appContext,
workerParams) {

    override suspend fun doWork(): Result {
        // Get the category passed from MainActivity (default
to technology)
        val category = inputData.getString("CATEGORY") ?:
"technology"

        return try {
            // Free mock API - no key needed!
            val urlString =
"https://saurav.tech/NewsAPI/top-headlines/category/$cat
egory/in.json"
            val url = URL(urlString)
            val connection = url.openConnection() as
HttpURLConnection
            connection.requestMethod = "GET"

            val response =
connection.inputStream.bufferedReader().use {
it.readText() }
            val jsonObject = JSONObject(response)

            // Get the very first article from the array
            val firstArticle =
jsonObject.getJSONArray("articles").getJSONObject(0)
            val title = firstArticle.getString("title")
            val source =
firstArticle.getJSONObject("source").getString("name")
            val articleUrl = firstArticle.getString("url") //  Grab
the URL

            // Trigger Notification (Pass the URL now)
            sendNotification("Breaking:
${category.uppercase()}", "$title\nSource: $source",
articleUrl)

            Result.success()
        } catch (e: Exception) {
            e.printStackTrace()
            Result.failure()
        }
    }

    private fun sendNotification(title: String, message:
String, articleUrl: String) {
        val manager =
applicationContext.getSystemService(Context.NOTIFICA
TION_SERVICE) as NotificationManager
        val channelId = "news_alerts"

        // Required for Android 8.0+
        if (Build.VERSION.SDK_INT >=
Build.VERSION_CODES.O) {
            val channel = NotificationChannel(channelId,
"News Alerts",
NotificationManager.IMPORTANCE_HIGH)
            manager.createNotificationChannel(channel)
        }

        // --- PENDING INTENT LOGIC FOR CLICK
ACTION ---
        val intent = Intent(applicationContext,
MainActivity::class.java).apply {
            flags = Intent.FLAG_ACTIVITY_NEW_TASK or
Intent.FLAG_ACTIVITY_CLEAR_TASK
            // STUFF THE URL INTO THE INTENT
PAYLOAD
            putExtra("ARTICLE_URL", articleUrl)
        }
```

```
        // FLAG_UPDATE_CURRENT is required to ensure
our new URL string is attached
        val pendingIntent: PendingIntent =
PendingIntent.getActivity(
            applicationContext,
            0,
            intent,
            PendingIntent.FLAG_UPDATE_CURRENT or
PendingIntent.FLAG_IMMUTABLE
        )

        val builder =
NotificationCompat.Builder(applicationContext,
channelId)
```

MainActivity.kt:

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        //  CATCH THE NOTIFICATION PAYLOAD!
        // If the user clicked the notification, this will grab the
URL and open the browser.
        val passedUrl =
intent.getStringExtra("ARTICLE_URL")
        if (passedUrl != null) {
            val browserIntent = Intent(Intent.ACTION_VIEW,
Uri.parse(passedUrl))
            startActivity(browserIntent)
        }

        // Permission check
        if (Build.VERSION.SDK_INT >=
Build.VERSION_CODES.TIRAMISU) {
            if (ContextCompat.checkSelfPermission(this,
Manifest.permission.POST_NOTIFICATIONS) !=
PackageManager.PERMISSION_GRANTED) {
                ActivityCompat.requestPermissions(this,
arrayOf(Manifest.permission.POST_NOTIFICATIONS),
102)
            }
        }

        val btnMonitor =
findViewById<Button>(R.id.btnMonitor)
        val tvStatus =
findViewById<TextView>(R.id.tvStatus)
        val chipGroup =
findViewById<ChipGroup>(R.id.chipGroup)

        btnMonitor.setOnClickListener {
.setSmallIcon(android.R.drawable.ic_menu_info_details)
            .setContentTitle(title)

.setStyle(NotificationCompat.BigTextStyle().bigText(mess
age))
            .setPriority(NotificationCompat.PRIORITY_HIGH)
            .setAutoCancel(true) // Dismisses alert when
tapped
            .setContentIntent(pendingIntent) // Connects the
tap action!

        manager.notify(2, builder.build())
    }
}

            tvStatus.text = "Status: Scanning for breaking
news..."

            // Determine which category to pass to the Worker
            val category = if (chipGroup.checkedChipId ==
R.id.chipBusiness) "business"
            else if (chipGroup.checkedChipId ==
R.id.chipScience) "science"
            else "technology"

            // Pass data to Worker
            val inputData =
Data.Builder().putString("CATEGORY", category).build()

            // Trigger Background Task
            val newsRequest =
OneTimeWorkRequestBuilder<NewsWorker>()
                .setInputData(inputData)
                .build()

WorkManager.getInstance(this).enqueue(newsRequest)

            // Simulate the B&W Card appearing after
fetching (for your printed screenshot!)
            createImpressiveBWCard(category)
        }
    }

    private fun createImpressiveBWCard(category: String)
{
        val container =
findViewById<LinearLayout>(R.id.newsContainer)
        container.removeAllViews() // Clear previous

        // 1. Create the Card
        val card = MaterialCardView(this)
        val cardParams =
LinearLayout.LayoutParams(ViewGroup.LayoutParams.
```

```
MATCH_PARENT,
ViewGroup.LayoutParams.WRAP_CONTENT)
        cardParams.setMargins(0, 16, 0, 16)
        card.layoutParams = cardParams
        card.setCardBackgroundColor(Color.WHITE)
        card.strokeColor = Color.BLACK
        card.strokeWidth = 4 // Thick brutalist border
        card.radius = 0f // Sharp corners for print

        // 2. Create the Layout inside the Card
        val innerLayout = LinearLayout(this)
        innerLayout.orientation = LinearLayout.VERTICAL
        innerLayout.setPadding(32, 32, 32, 32)

        // 3. Add Headline
        val title = TextView(this)
        title.text = when (category) {
            "business" -> "Global Markets Fluctuate Amid
Policy Changes"
            "science" -> "Astronomers Discover New
Exoplanet in Habitable Zone"
            else -> "New AI Model Outperforms Humans in
Logic Tests"
        }
        title.textSize = 20f
        title.setTextColor(Color.BLACK)
        title.setTypeface(null,
android.graphics.Typeface.BOLD)

        // 4. Add Source/Metadata
        val metadata = TextView(this)
        metadata.text = "Source: The Global Times | Just
Now"
        metadata.textSize = 12f
        metadata.setTextColor(Color.DKGRAY)
        metadata.setPadding(0, 16, 0, 0)

        // Assemble
        innerLayout.addView(title)
        innerLayout.addView(metadata)
        card.addView(innerLayout)
        container.addView(card)
    }
}
```

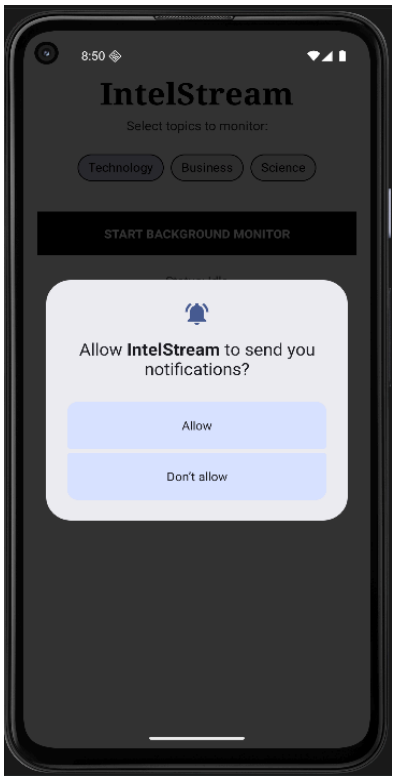
activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/andr
oid"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#FFFFFF"
    android:fitsSystemWindows="true"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="IntelStream"
        android:textAlignment="center"
        android:textColor="#000000"
        android:fontFamily="serif"
        android:textSize="36sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="4dp"
        android:text="Select topics to monitor:"
        android:textAlignment="center"
        android:textColor="#333333"
        android:textSize="15sp" />

    <com.google.android.material.chip.ChipGroup
        android:id="@+id/chipGroup"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center_horizontal"
        android:layout_marginTop="16dp"
        app:singleSelection="true">

        <com.google.android.material.chip.Chip
            android:id="@+id/chipTech"

style="@style/Widget.MaterialComponents.Chip.Choice"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Technology"
            android:textColor="#000000"
            android:checked="true"
            app:chipStrokeColor="#000000"
            app:chipStrokeWidth="1dp" />

        <com.google.android.material.chip.Chip
            android:id="@+id/chipBusiness"

style="@style/Widget.MaterialComponents.Chip.Choice"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Business"
```

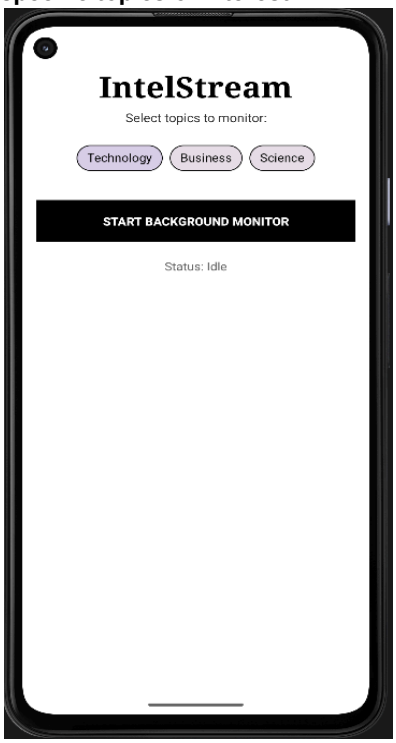
```
            android:textColor="#000000"
            app:chipStrokeColor="#000000"
            app:chipStrokeWidth="1dp" />

        <com.google.android.material.chip.Chip
            android:id="@+id/chipScience"

style="@style/Widget.MaterialComponents.Chip.Choice"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Science"
            android:textColor="#000000"
            app:chipStrokeColor="#000000"
            app:chipStrokeWidth="1dp" />

    </com.google.android.material.chip.ChipGroup>

    <Button
        android:id="@+id/btnMonitor"
        android:layout_width="match_parent"
        android:layout_height="60dp"
        android:layout_marginHorizontal="16dp"
        android:layout_marginTop="24dp"
        android:backgroundTint="#000000"
        android:text="START BACKGROUND MONITOR"
        android:textColor="#FFFFFF"
        android:textStyle="bold"
        app:cornerRadius="0dp" />

    <TextView
        android:id="@+id/tvStatus"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Status: Idle"
        android:textAlignment="center"
        android:textColor="#666666" />

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:layout_marginHorizontal="8dp"
        android:layout_marginTop="16dp">

        <LinearLayout
            android:id="@+id/newsContainer"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical" />
    </ScrollView>

</LinearLayout>
```

OUTPUT:

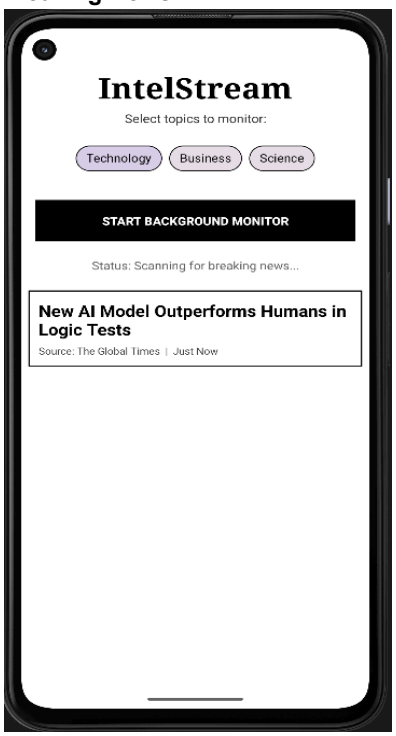
Allow to send notifications

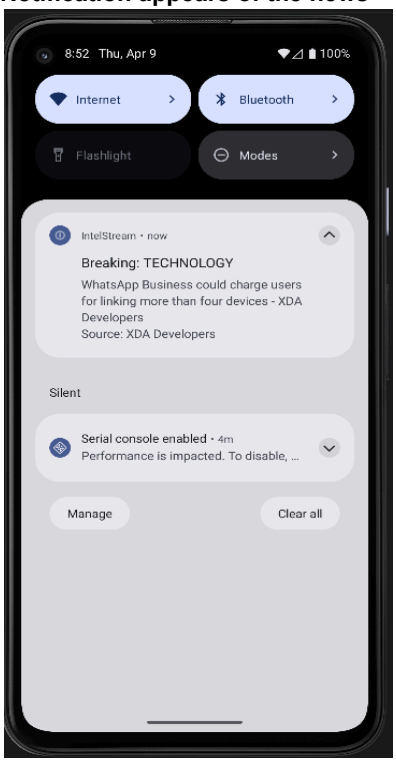


Start background monitor for news on specific topics of interest



Breaking News



Go back to home screen	Notification appears of the news	Tap on notif to redirect to news article
		

CONCLUSION:

This experiment demonstrated the critical importance of decoupling network operations from the main UI thread using modern Jetpack architecture. By successfully implementing a dynamic WorkManager that passes user-defined parameters to a background REST API call, and tying the response to a high-priority NotificationChannel, the application proved its ability to deliver personalized, real-time content. This approach highlights the standard industry practices required to build highly engaging, responsive, and battery-conscious applications that keep users informed even when the software is not actively in use.